

HIGH PERFORMANCE COMPUTING FOR SCIENCE AND ENGINEERING

Harvard University
CS 2050

Lecture 7
February 17, 2026

SETUP

- Request 12 cores on a compute node

```
srun --nodes=1 --ntasks=12 --time=02:00:00 --pty bash
```

- Load the CS 2050 environment

```
. ~/161588/spack/share/spack/setup-env.sh  
spack env activate -p CS-2050
```

QUIZ 3



Account



Dashboard



Courses



Calendar



Inbox



History



Help

COMPSCI 2050 > Quizzes > Quiz 3

2025-2026 Spring

- Home
- Announcements
- Course Calendar
- Q & A
- Zoom
- DCE Class Recordings
- Quizzes**
- Gradescope
- People
- Grades
- Files

Quiz 3

Due Feb 17 at 11:59pm **Points** 13 **Questions** 13
Available Feb 16 at 12pm - Jul 1 at 11:59pm **Time Limit** 60 Minutes

Instructions

The quizzes are designed to take no more than 30 minutes, but one hour will be allowed.

Do not click into the quiz until you are ready to take it.

Take the Quiz

HOMework 2

 CS-2050 / homework-2 

[Code](#) [Issues](#) [Pull requests](#) [Actions](#) [Projects](#) [Wiki](#) [Security](#) [Insights](#) [Settings](#)

 **homework-2** Internal template [Edit Pins](#) [Watch 0](#) [Fork 0](#)

[main](#) [1 Branch](#) [0 Tags](#) [Add file](#) [Code](#)

 **laz380 and Chuck Witt** public release.

b5effe · 4 days ago  1 Commits

 question-1	public release.	4 days ago
 question-2	public release.	4 days ago
 question-3	public release.	4 days ago
 question-4	public release.	4 days ago
 question-5	public release.	4 days ago
 question-6	public release.	4 days ago
 question-7	public release.	4 days ago
 .gitignore	public release.	4 days ago
 README.md	public release.	4 days ago

CLUSTER A BIT SLUGGISH RECENTLY



HARVARD UNIVERSITY
Information Technology

OnDemand provides an integrated, single access point for all of your HPC resources.

Latest News for HUIT Open OnDemand

Current status: Slow performance

We are aware of ongoing performance issues with HUIT Open OnDemand. You may notice:

- The Dashboard is slow to refresh or load
- Interactive Apps (including new sessions) can be slow to start or display

Our team is investigating and working to restore normal performance.



main

1 Branch 0 Tags

Go to file

t

Add file

<> Code



wcw398

eb68eec · now

6 Commits



lecture-02

add lecture-02.

2 weeks ago



lecture-04

lecture-04 examples.

2 weeks ago



lecture-06

.

5 days ago



README.md

.

now



README



Useful Commands

Request 12 tasks on a compute node for interactive use

```
srun --nodes=1 --ntasks=12 --time=02:00:00 --pty bash
```



Load the CS 2050 environment

```
. ~/161588/spack/share/spack/setup-env.sh  
spack env activate -p CS-2050
```



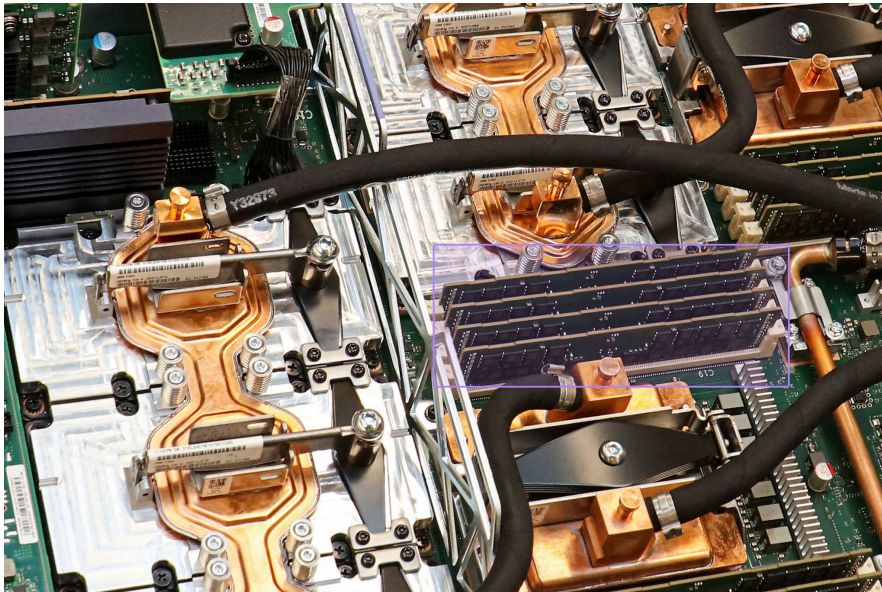
TOPICS

Main topic: **Distributed memory programming with MPI**

- Why distributed memory parallelism?
- Distributed vs shared memory models
- Introduction to MPI and its history
- MPI communicators and groups
- Point-to-point and collective communication
- Synchronous and asynchronous communication
- Anatomy of a MPI message
- Getting started with MPI and first examples

WHY DISTRIBUTED PARALLELISM?

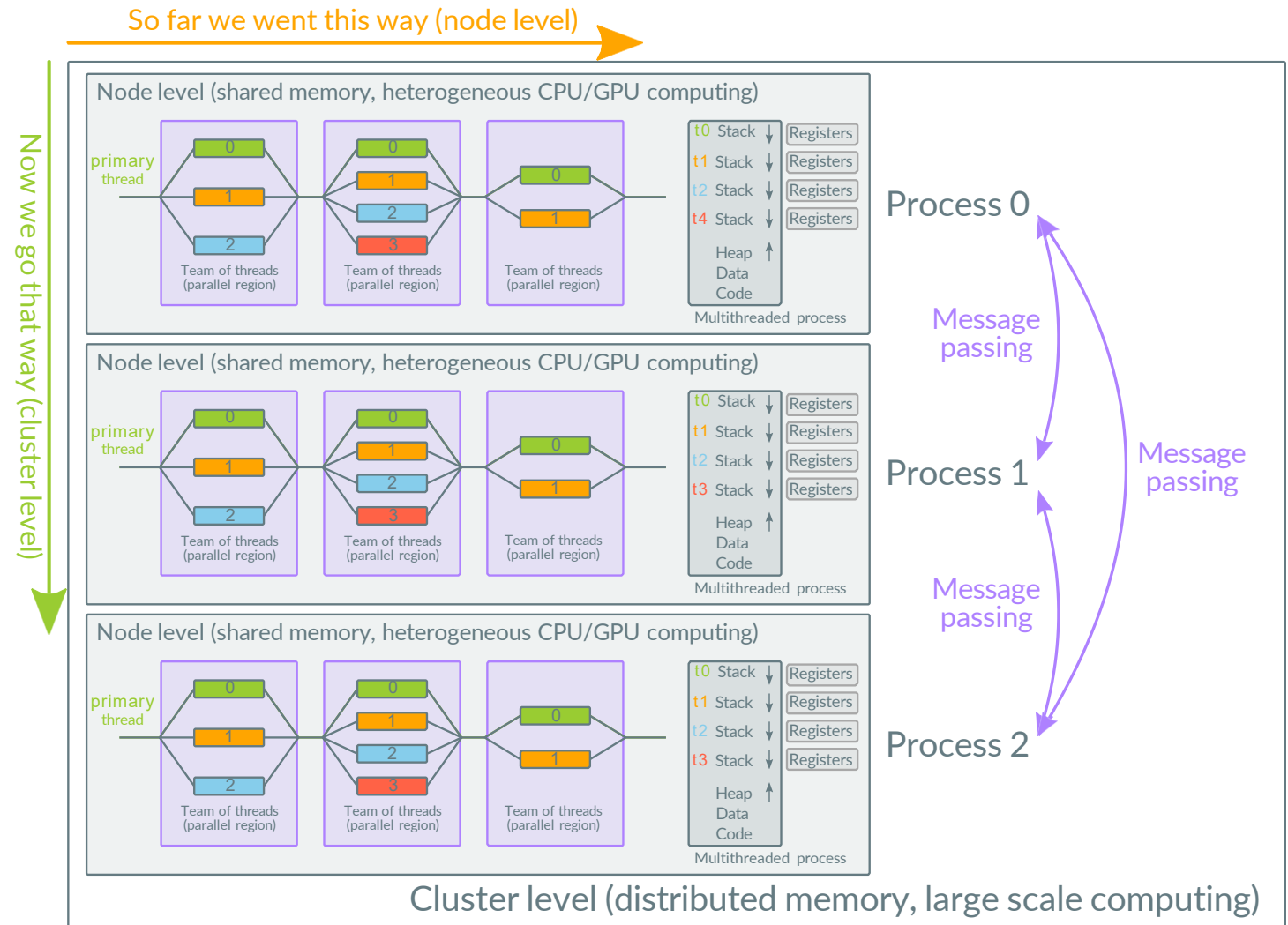
- Maximum memory per node is limited
- For large computing tasks, single-node memory is simply not enough
 - fluid dynamics
 - molecular dynamics
 - machine learning
 - large scale data processing



- **Solution:** Many nodes connected by (HPC) network
- Memory is *distributed* among connected nodes. Communication achieved by *sending* or *receiving* messages over the network.

ORTHOGONALITY

- Shared memory and distributed memory are two orthogonal programming models



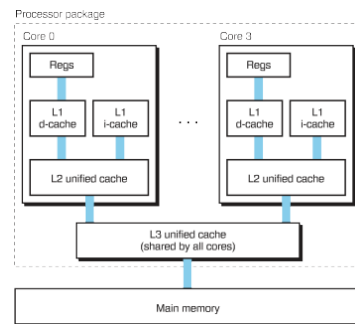
SHARED VERSUS DISTRIBUTED MEMORY

Shared memory:

- Accessing data is easy
- Race conditions
- False sharing

On-Chip Network (OCN):

- **Fast** communication
- Maximum distance **few centimeters**
- Examples: memory bus, [Intel Ultra Path](#)

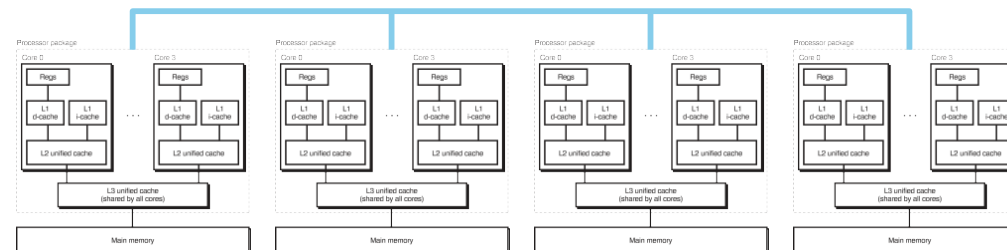


Distributed memory:

- No direct access to distributed data
- You must take care of communication *explicitly*
- No race conditions for distributed communication
- Can scale to much larger number of parallel processors

System Area Network (SAN):

- **Slow** communication
- Examples: [InfiniBand](#), [Cray Slingshot](#) or [Ethernet](#)



RECALL: PROCESSES AND THREADS

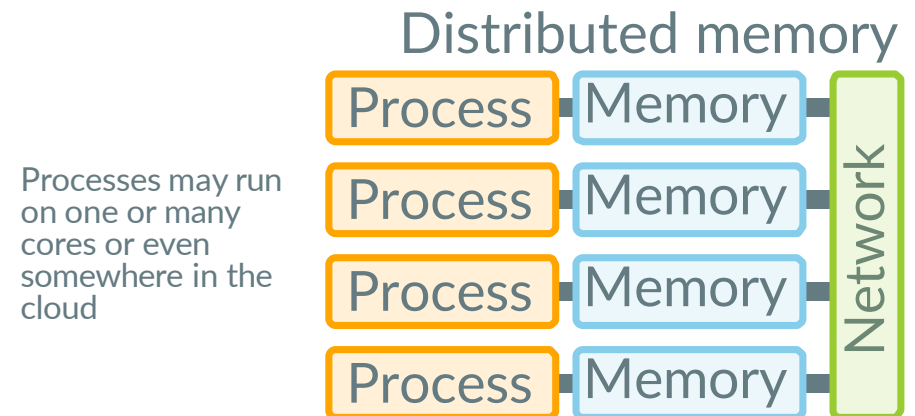
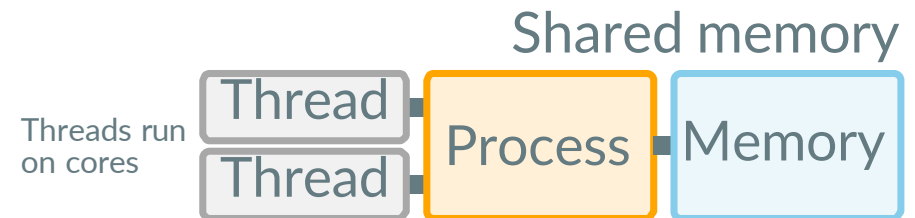
Processes:

OS creates processes - expensive to create. Each process has its own virtual address space.

Logical (software) threads:

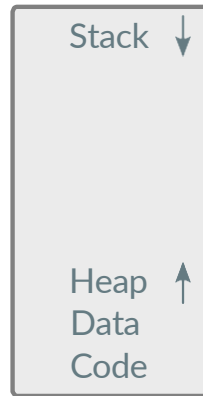
Threads share the resources of a process. Less expensive to create.

- Does *not* mean each process must live in a separate physical memory space (or on different nodes)
- For example, many processes run simultaneously on your laptop
Does not matter whether these processes originate from individual programs or from a distributed memory code



MESSAGE PASSING INTERFACE (MPI)

- A process cannot access the memory of another process. (Basic safety requirement in multi-user environments such as Linux)
- Communication between processes is important when they cooperate on a problem (e.g. MPI)
- Inter-process communication is implemented by the OS with several mechanisms: signals, files, pipes, sockets or shared memory



Simplified representation of the virtual address space of a Linux process

- Many different MPI *implementations* of the *specification* exist. Major ones are:
 - OpenMPI (<https://www.open-mpi.org/>)
 - MPICH (<https://www.mpich.org/>)
 - MVAPICH (<https://mvapich.cse.ohio-state.edu/>)
- Optimized vendor specific implementations exist for Cray/HPE, Intel, IBM, Microsoft, etc.

- Should we write our own protocols to communicate between processes? **No!** Would be error-prone, and overkill
- Since such communication is ubiquitous, a *standard* exists:
the *Message Passing Interface (MPI)*

MPI specifies an API
(Application Programming Interface)

- The MPI code you write does not depend on a specific *implementation*. Your code will run on a compute cluster just like it will run on your laptop.
- All you need is to include the API declarations in your code:

```
1 #include <mpi.h>
```

MPI HISTORY

- In the 1980's to the early 1990's, parallel computing with distributed memory required a large number of incompatible software tools for writing such programs. The need for portability, performance, functionality and price. Recognition of the need for a standard.
- Working group meets in Minneapolis. MPI draft proposal (MPI-1) is developed. The group adopts procedures and organization to form the MPI Forum. It consists of individuals from 40 organizations including parallel computer vendors, system integrators and application scientists.

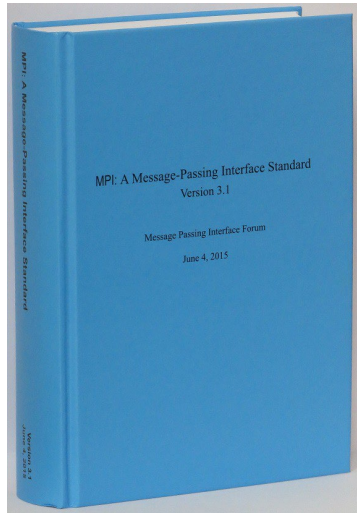
MPI 5.0

MPI-5.0 was approved by the MPI Forum on June 5, 2025.

- [MPI 5.0 document as PDF](#)
- [BibTeX file for citing MPI 5.0](#)
- [Unofficial HTML version of MPI 5.0](#)

Version	Year	Description
1.0	1994	The MPI Forum with participation from over 40 organizations releases the first standard for a message passing interface. The MPI Forum is not sanctioned or supported by any official standards organization.
2.0	1997	More focus on process creation, added one-sided communications, extended collective communications and added parallel I/O.
3.0	2012	Significant extensions to MPI functionality including non-blocking collectives, new one-sided communication operations and Fortran 2008 bindings. This standard is a major update to earlier versions.
4.0	2021	Major update with significant new functionality including persistent collectives, partitioned communications and improvements to error handling.

MPI STANDARD



- ♦ Full documentation with useful examples
- ♦ This is the main MPI reference

- ♦ MPI-4.0: <https://www.mpi-forum.org/docs/mpi-4.0/mpi40-report.pdf> (also in the class git repo)
- ♦ MPI-3.1: <https://www.mpi-forum.org/docs/mpi-3.1/mpi31-report.pdf>

Practice is most important:

- <https://hpc-tutorials.llnl.gov/mpi>
- <https://www.mcs.anl.gov/research/projects/mpi/tutorial>
- <https://mpitutorial.com>

Another option for docs are the man-pages

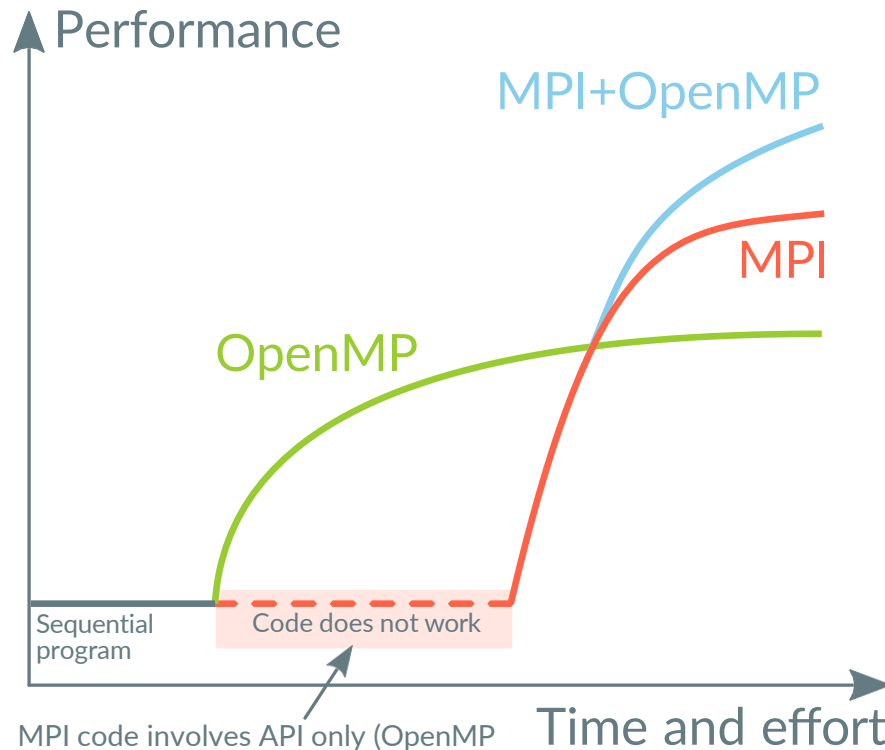
- A complete MPI installation includes the docs and man-pages.
- Invaluable when you work in the command line.
- When you have loaded a MPI module on the cluster you can query the man-pages to get instant help for any MPI routine

For example:

```
1 man MPI_Init
2 MPI_Init(3)          Open MPI          MPI_Init(3)
3
4 NAME
5     MPI_Init - Initializes the MPI execution environment
6
7 SYNTAX
8 C Syntax
9     #include <mpi.h>
10     int MPI_Init(int *argc, char ***argv)
11 ...
```

OVERHEAD BETWEEN OPENMP AND MPI

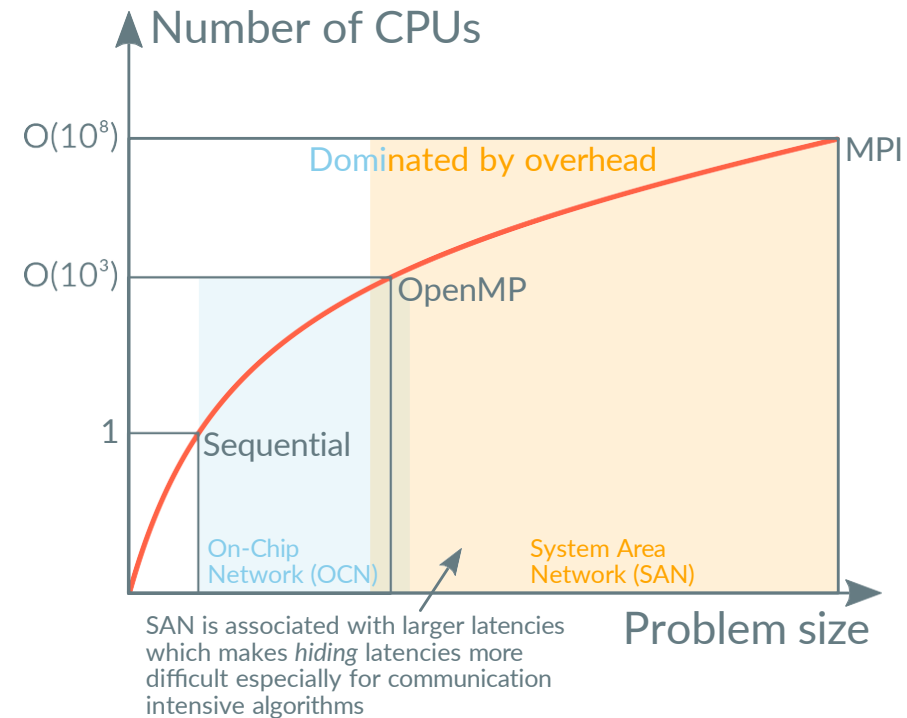
Time to (parallel) software:



MPI code involves API only (OpenMP is mainly compiler directives and some runtime routines). This overhead is mostly due to *explicit communication*.

Code does not work in initial parallelization phase because communication must be handled *explicitly* by API calls.

Communication overhead:



A SAN can cover larger area at the cost of slower speed (same story with caches, cell phones, etc.) and therefore higher latencies. Hiding these latencies is more difficult the more complex the system becomes.

WHY USE MPI?

Standardization

MPI is the only message passing library that can be considered a standard. It is supported on virtually all HPC platforms.

Portability

There is little or no need to modify your source code when you port your application to a different platform that supports (and is compliant with) the MPI standard.

Performance opportunities

Vendor implementations should be able to exploit native hardware features to optimize performance. Any implementation is free to develop optimized algorithms.

Functionality

There are over 430 routines defined in MPI-3, which includes the majority of those in MPI-2 and MPI-1.

Note: Most MPI programs can be written using a dozen or fewer routines.

Availability

A variety of implementations are available, both vendor and public domain.

MPI COMMUNICATORS AND GROUPS

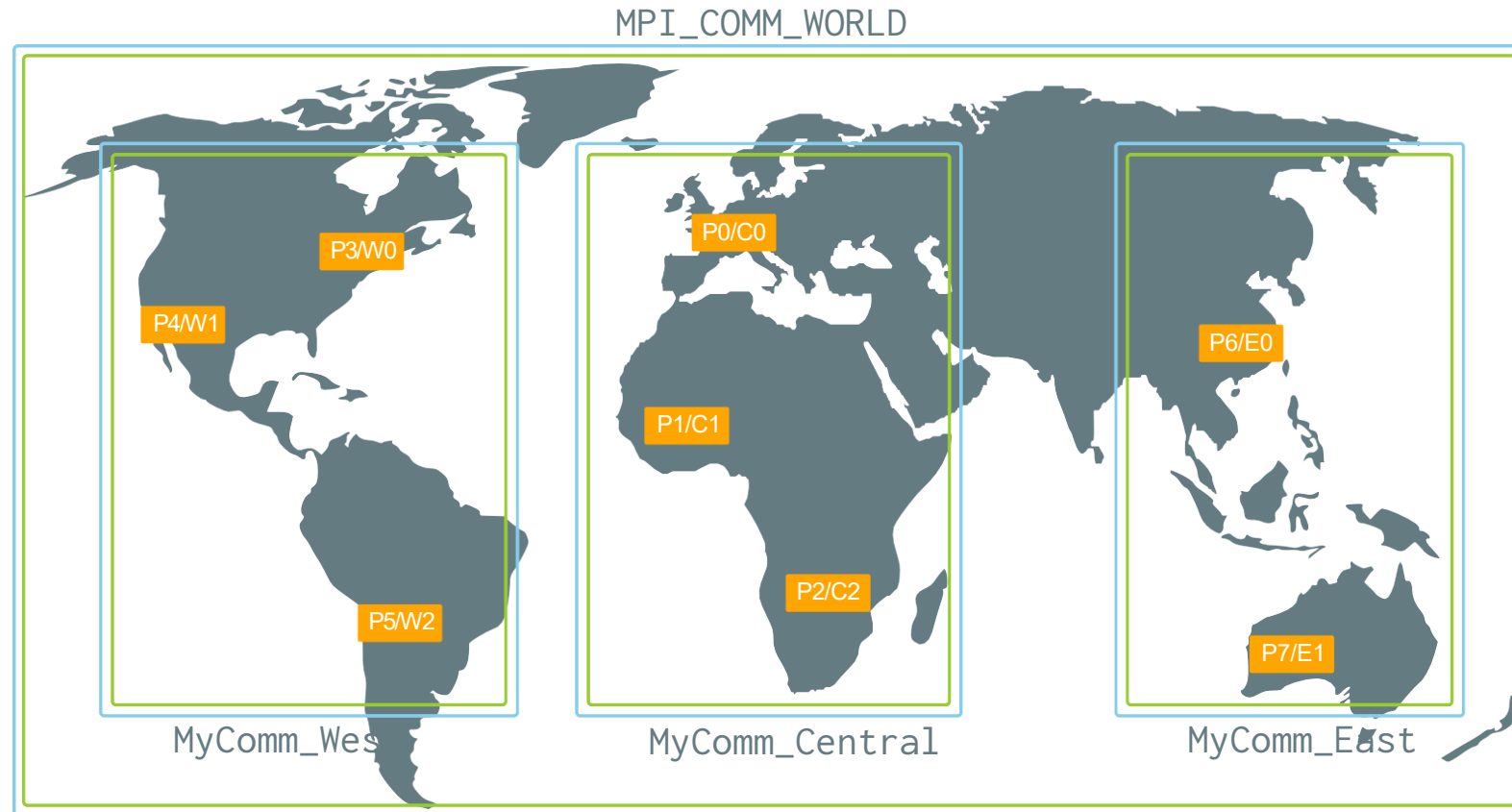
- MPI uses objects called *communicators* and *groups* to define which collection of *processes* may communicate with each other.
- Most MPI routines require you to specify a *communicator as an argument*.
- `MPI_COMM_WORLD` is the predefined default communicator that includes all of your MPI processes.

An MPI communicator encapsulates:

- *Contexts* (the communication "universe")
- *Groups* (the *processes* that make up the universe, or a subset of it)
- *Virtual topologies* (e.g. arrange the processes on a Cartesian grid.)



MPI COMMUNICATORS AND GROUPS

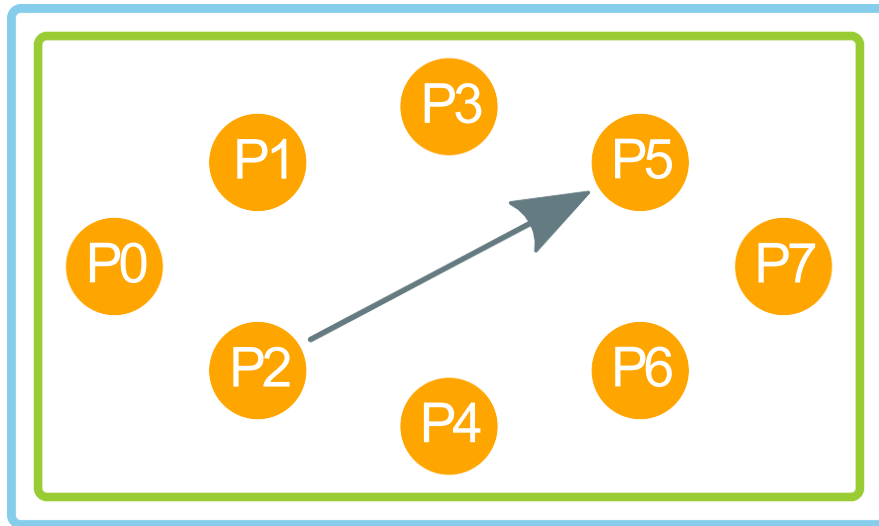


- A MPI *group* is an ordered set of processes.
- Each process is associated with an integer *rank*. Ranks are *contiguous* and start from zero.
- A group is used within a communicator to describe the participants in a communication "universe".
- Communicators can be split into *sub-communicators*. E.g., MPI_COMM_WORLD is split into MyComm_West, MyComm_Central and MyComm_East above.

POINT-TO-POINT AND COLLECTIVE COMMUNICATION

The fundamental operation in MPI is point-to-point communication:

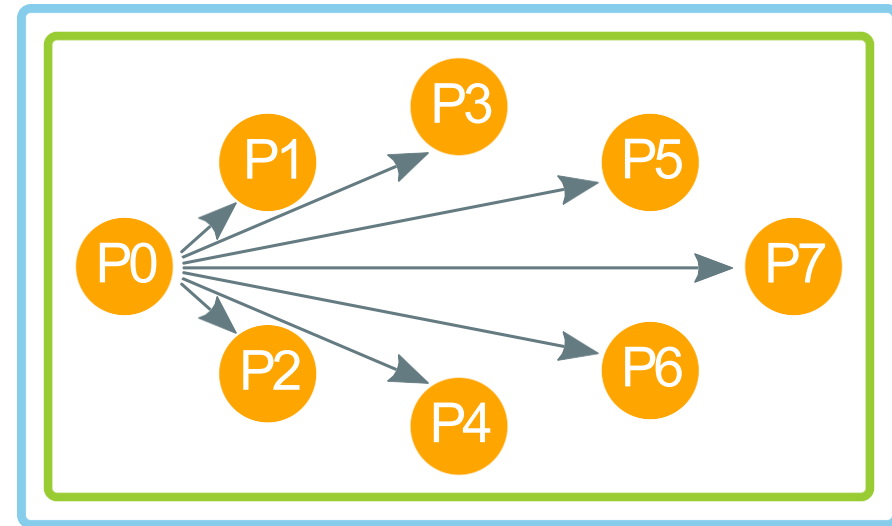
MPI_COMM_WORLD



- It is simply a *matching* send/receive pair.
- Think of "point" as a process within a group.
- The two points must be in the same group

Collective communication involves ALL ranks in a group:

MPI_COMM_WORLD



- Any collective communication can be composed with point-to-point communication only.
 - They exist for convenience and *performance* reasons.
 - Collective primitives are generally *expensive*.
- Examples:**
- Barrier
 - Broadcast
 - Reduction
 - Scan

SYNCHRONOUS AND ASYNCHRONOUS COMMUNICATION

MPI specifies two communication behaviors:

Blocking

When the API function call for a blocking primitive returns, MPI *guarantees* that it is safe to use the message buffer again (for sender and/or receiver).

Non-Blocking

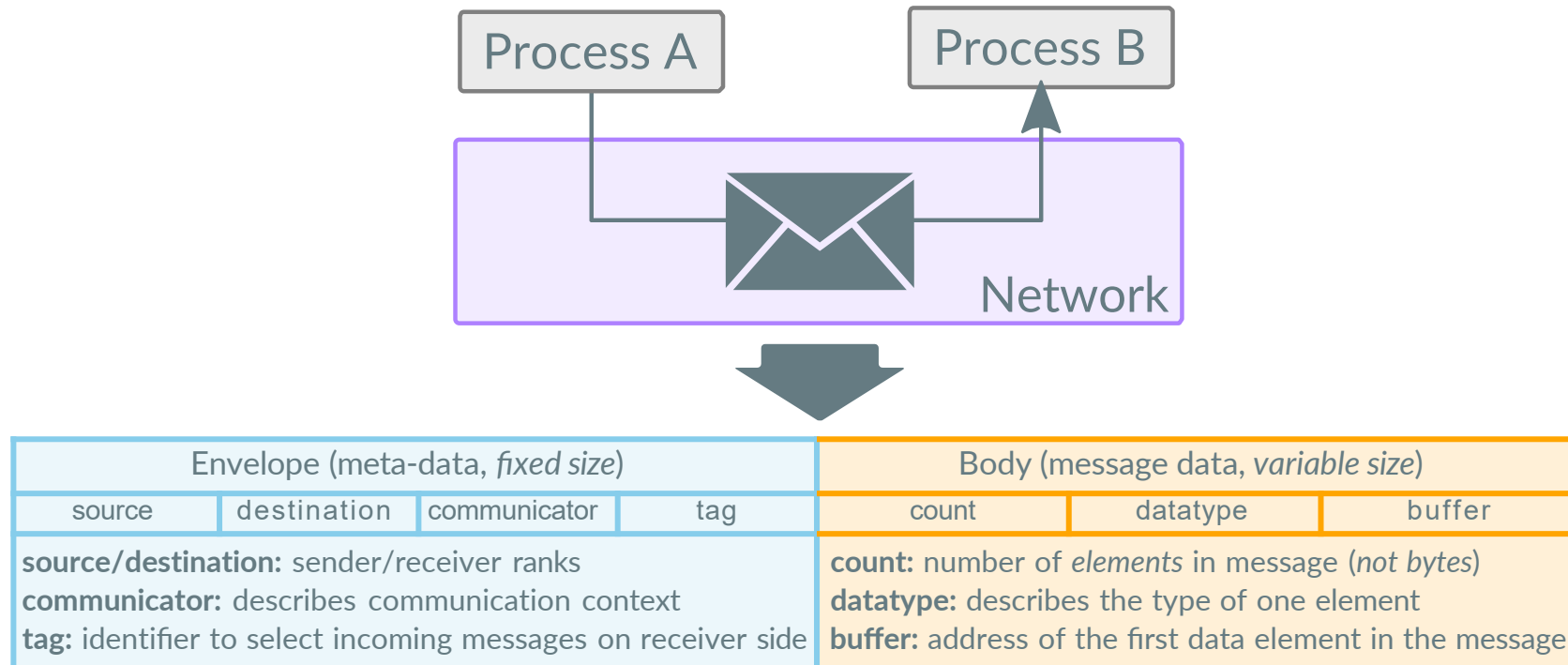
The API function call for a non-blocking primitive *returns immediately*. There is *no guarantee* that the communication has concluded. Communication buffers *are not* safe for read/write operations.

Blocking communication means synchronization which results in sequential code!

A blocked process does not make progress (it does not perform useful work while blocked). Non-blocking communication allows to do work while messages are in flight.

Most useful property of non-blocking communication is compute-transfer overlap.

ANATOMY OF AN MPI MESSAGE



Example:

```
1 if (Process_A) {  
2     // destination = Process B  
3     MPI_Send(send_buffer, count, datatype, destination, tag, MPI_COMM_WORLD);  
4 } else if (Process_B) {  
5     // source = Process A  
6     MPI_Recv(recv_buffer, count, datatype, source, tag, MPI_COMM_WORLD, MPI_STATUS_IGNORE);  
7 }
```

General MPI program structure:

```
1 // headers and declarations
2 #include <mpi.h> // includes the MPI API
3 // other headers and declarations
4 int main(int argc, char *argv[]) // Program start
5 {
6     // 1. Serial code
7
8     // 2. Initialize MPI environment
9     MPI_Init(&argc, &argv); // parallel code begins
10
11     // 3. Do work and MPI calls here
12
13     // 4. Terminate MPI environment
14     MPI_Finalize(); // parallel code ends
15
16     // 5. Serial code
17
18     return 0;
19 } // Program end
```

- It is *necessary* to initialize and terminate the MPI environment.
- MPI calls can only be carried out after initialization and before termination of the MPI environment.
- Before initialization and after termination of the MPI environment the code is serial (i.e., you cannot use MPI).
- You must include the `mpi.h` header to make use of MPI library calls.
- The names of the MPI routines all start with `MPI_` (or `PMPI_` for the profiling interface) and *are case-sensitive* in C/C++ code (they are not in Fortran).
- Your code ***must not*** declare variables or functions that begin with `MPI_` or `PMPI_`.

HELLO MPI

Hello MPI:

```

1  #include <iostream>
2  #include <mpi.h>
3
4  int main(int argc, char *argv[])
5  {
6      MPI_Init(&argc, &argv);
7
8      std::cout << "Hello MPI\n";
9
10     MPI_Finalize();
11     return 0;
12 }

```

MPI compiler wrapper for C++:

```

1  $ mpic++ -show # output shortened
2  g++ -pthread -L/usr/lib/openmpi -lmpi

```

Compilation is similar as with g++:

```

1  $ mpic++ -o hello_mpi hello_mpi.cpp

```

MPI runtime wrapper:

```

1  $ mpirun -n 2 ./hello_mpi
2  Hello MPI
3  Hello MPI

```

- The `-n 2` argument creates 2 copies of the program.
- We can check that this creates 2 processes:

```

1  $ ps aux | grep hello_mpi
2  fabs  584178  20:00  0:00 mpirun -n 2 ./hello_mpi
3  fabs  584180  20:00  0:00 ./hello_mpi
4  fabs  584181  20:00  0:00 ./hello_mpi

```

Running MPI on SLURM:

```

$ srun -n2 -c1 ./hello_mpi
Hello MPI
Hello MPI

```

- SLURM uses `srun` to run parallel jobs.
- You have already used it in your job scripts. The option `-n2` requests 2 tasks (`--ntasks`) where each task has `-c1` cores to run on (`--cpus-per-task`).

MPI ENVIRONMENT MANAGEMENT ROUTINES

MPI_Init and **MPI_Init_thread**

Initializes the MPI execution environment.

MPI_Init must be called in every MPI program, must be called before any other MPI functions and must be called only once.

For thread support you must initialize the environment with MPI_Init_thread.

MPI_Comm_size

Returns the total number of MPI processes in the specified communicator, such as MPI_COMM_WORLD.

MPI_Comm_rank

Returns the rank of the calling MPI process within the specified communicator. Initially, each process will be assigned a unique integer rank between 0 and number of tasks - 1 within the communicator MPI_COMM_WORLD.

MPI_Abort

Terminates all MPI processes associated with the communicator. In most MPI implementations it terminates *all* processes regardless of the communicator specified.

MPI_Wtime

Returns an elapsed wall clock time in seconds (double precision) on the calling processor.

MPI_Get_version

Returns the version and subversion of the MPI standard that is implemented by the library.

MPI_Finalize

Terminates the MPI execution environment. This function should be the last MPI routine called in every MPI program—no other MPI routines may be called after it.



PRINT HOSTNAME WITH MPI

```
1 #include <cassert>
2 #include <iostream>
3 #include <mpi.h>
4
5 int main(int argc, char *argv[])
6 {
7     int size, rank, len, err;
8     char hostname[MPI_MAX_PROCESSOR_NAME];
9
10    // initialize MPI
11    err = MPI_Init(&argc, &argv);
12    assert(err == MPI_SUCCESS);
13
14    // get number of ranks in the MPI_COMM_WORLD communicator
15    err = MPI_Comm_size(MPI_COMM_WORLD, &size);
16    assert(err == MPI_SUCCESS);
17
18    // get my rank
19    err = MPI_Comm_rank(MPI_COMM_WORLD, &rank);
20    assert(err == MPI_SUCCESS);
21
22    // query processor name (the return value is
23    // implementation dependent)
24    err = MPI_Get_processor_name(hostname, &len);
25    assert(err == MPI_SUCCESS);
26    std::cout << "Rank " << rank << "/" << size <<
27              << " running on: " << hostname << '\n';
28
29    // call other MPI routines
30
31    // terminate the MPI environment
32    err = MPI_Finalize();
33    assert(err == MPI_SUCCESS);
34
35    return 0;
36 }
```

- The code uses `MPI_Comm_size` and `MPI_Comm_rank` to determine the number of ranks and the process rank (or task ID) in the `MPI_COMM_WORLD` communicator.
- Almost all MPI routines return an error code. It is good practice to check it (but often skipped when you read other code).

```
1 $ make
2 mpic++ -O1 -Wall -o rank_size_proc rank_size_proc.cpp
3 $ mpirun -n 4 ./rank_size_proc
4 Rank 0/4 running on: pilatus
5 Rank 1/4 running on: pilatus
6 Rank 3/4 running on: pilatus
7 Rank 2/4 running on: pilatus
```

- Similar as in shared memory, the order of the output from each process is arbitrary.

MPI WITH THREAD SUPPORT

```

1  #include <iostream>
2  #include <mpi.h>
3
4  int main(int argc, char *argv[])
5  {
6      int rank, size, version, subversion, provided;
7      MPI_Init_thread(&argc, &argv, MPI_THREAD_FUNNELED, &provided);
8
9      MPI_Get_version(&version, &subversion);
10     MPI_Comm_rank(MPI_COMM_WORLD, &rank);
11     MPI_Comm_size(MPI_COMM_WORLD, &size);
12     const bool isroot = (0 == rank);
13     if (isroot) { // only the root rank executes this code
14         std::cout << "MPI version:      " << version << '.' << subversion
15                     << '\n';
16         std::cout << "Total ranks:      " << size << '\n';
17         std::cout << "Thread support:   " << provided
18                     << " (requested MPI_THREAD_FUNNELED)\n";
19         std::cout << "MPI_THREAD_SINGLE: " << MPI_THREAD_SINGLE << '\n';
20         std::cout << "MPI_THREAD_FUNNELED: " << MPI_THREAD_FUNNELED << '\n';
21         std::cout << "MPI_THREAD_SERIALIZED: " << MPI_THREAD_SERIALIZED << '\n';
22         std::cout << "MPI_THREAD_MULTIPLE: " << MPI_THREAD_MULTIPLE << '\n';
23     }
24
25     MPI_Finalize();
26     return 0;
27 }

```

Output:

```

1 MPI version:      3.1
2 Total ranks:      4
3 Thread support:   1 (requested MPI_THREAD_FUNNELED)
4 MPI_THREAD_SINGLE: 0
5 MPI_THREAD_FUNNELED: 1
6 MPI_THREAD_SERIALIZED: 2
7 MPI_THREAD_MULTIPLE: 3

```

- If your MPI application is multi-threaded you need to initialize MPI with thread support using `MPI_Init_thread`.
- There are four possible levels:
 1. `MPI_THREAD_SINGLE`: Only one thread will execute.
 2. `MPI_THREAD_FUNNELED`: If the process is multithreaded, only the thread that called `MPI_Init_thread` will make MPI calls.
 3. `MPI_THREAD_SERIALIZED`: If the process is multi-threaded, only one thread will make MPI library calls at one time.
 4. `MPI_THREAD_MULTIPLE`: If the process is multithreaded, multiple threads may call MPI at once with no restrictions.

RECAPPING SO FAR ...

- MPI has a long history and is the *industry standard* for distributed memory programming.
- MPI processes are arranged in groups with unique ranks (task IDs).
- The default intra-communicator in MPI is called MPI_COMM_WORLD.
- Must initialize the MPI environment in your code
- All MPI routines have the following structure MPI_Xxxx and are case sensitive.

Now: ***Blocking MPI communication***

Details:

- MPI operations and MPI procedures
- Blocking point-to-point MPI communication
- Blocking collective MPI communication;
Examples of collective procedures

MPI OPERATIONS

- Sending and receiving messages is the basic MPI communication mechanism.
- An *MPI operation* is a sequence of steps performed by the MPI library to establish and enable data transfer and/or synchronization.

This sequence consists of *four* stages:

Initialization

Hands over the argument list to the *operation* but not the content of the data buffers (if any).

Starting

Hands over the control of the data buffers (if any) to the associated operation. **Note:** *initiation* refers to the combination of the initialization and starting stages.

Completion

Returns control of the content of the data buffers and indicates that the output buffers and arguments (if any) have been updated. **Note:** an MPI operation is *complete* when the MPI procedure implementing the completion returns.

Freeing

Returns control of the rest of the argument list (e.g., the data buffer address and array arguments).

- MPI *operations* are available in one or more of these forms: ***blocking***, ***non-blocking*** and ***persistent***.



FORMS OF MPI OPERATIONS

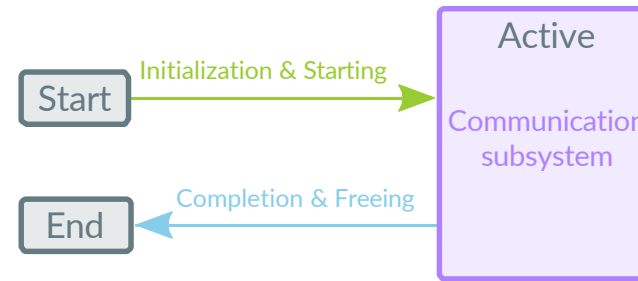
Blocking operation

For a **blocking operation**, all four stages are combined in a *single procedure call*.



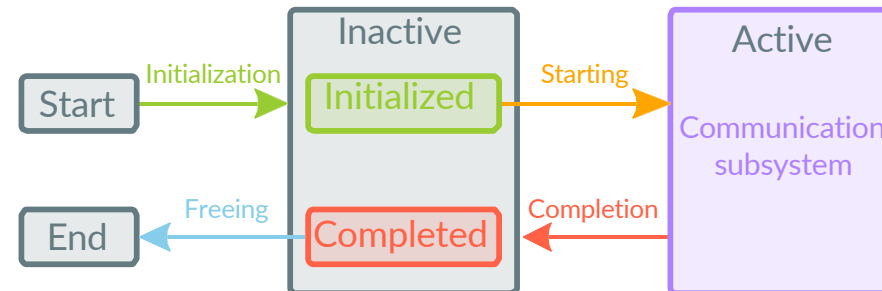
Non-blocking operation

For a **non-blocking operation**, the initialization and starting stages are combined into a single non-blocking procedure call and the completion and freeing stages are combined into a separate, single procedure call, which can be blocking or non-blocking.



Persistent operation

For a **persistent operation**, there is a separate procedure for each of the four stages. Each of these stages may be blocking or non-blocking. Persistent operations will not be part of CS 2050.



Additionally, an MPI operation can be **collective** or **non-collective**

BLOCKING POINT-TO-POINT COMMUNICATION

- Standard (blocking) sending and receiving of messages in MPI is achieved with `MPI_Send` and `MPI_Recv`, respectively.
- They are both *non-local* procedures. An `MPI_Send` *requires* a matching `MPI_Recv` to complete and vice versa.

```
1 int MPI_Send(  
2     const void *buf,          // message body  
3     int count,                // message body  
4     MPI_Datatype datatype,    // message body  
5     int dest,                 // message envelope  
6     int tag,                  // message envelope  
7     MPI_Comm comm             // message envelope  
8 )
```

```
1 int MPI_Recv(  
2     void *buf,                // message body  
3     int count,                // message body  
4     MPI_Datatype datatype,    // message body  
5     int source,               // message envelope  
6     int tag,                  // message envelope  
7     MPI_Comm comm,            // message envelope  
8     MPI_Status *status  
9 )
```

- An `MPI_Send` ***matches*** an `MPI_Recv` *only if* `dest`, `source`, `tag` and `comm` match.
- The return value is an error code. See `man MPI` for a full list of the codes. If the procedure is successful, it returns `MPI_SUCCESS`.
- To receive messages with arbitrary tags use `MPI_ANY_TAG` and to receive from any source use `MPI_ANY_SOURCE`.
- If the tag or source is unknown, use the `status` object to query the meta data.
- Often the status is not needed, use `MPI_STATUS_IGNORE` in this case.

MPI TYPE MATCHING AND TYPE CONVERSION

MPI_Datatype	C type
MPI_CHAR	char
MPI_SHORT	short int
MPI_INT	int
MPI_LONG	long int
MPI_LONG_LONG_INT	long long int
MPI_LONG_LONG	long long int
MPI_UNSIGNED_CHAR	unsigned char
MPI_UNSIGNED_SHORT	unsigned short int
MPI_UNSIGNED_INT	unsigned int
MPI_UNSIGNED_LONG	unsigned long int
MPI_UNSIGNED_LONG_LONG	unsigned long long int
MPI_FLOAT	float
MPI_DOUBLE	double
MPI_LONG_DOUBLE	long double
MPI_BYTE	
MPI_PACKED	

See Table 3.2 in MPI 4.0 standard

- MPI supports *heterogeneous* environments. For example, you can communicate between Fortran and C codes.
- When you write MPI code, you must use these data types for the message data in the body.
- MPI will not convert types. This send/receive pair will not work:

```
1 MPI_Send(buf, 1, MPI_INT, 1, 99, MPI_COMM_WORLD);
2 MPI_Recv(buf, 1, MPI_FLOAT, 0, 99, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
```
- MPI allows you to specify your own types as well, see Chapter 5 in the MPI 4.0 standard.

example-4

BLOCKING POINT-TO-POINT EXAMPLE

```
1 #include <mpi.h>
2 #include <iostream>
3
4 // exchange rank IDs between 2 processes
5 int main(int argc, char* argv[])
6 {
7     int rank, recv;
8     MPI_Init(&argc, &argv);
9     MPI_Comm_rank(MPI_COMM_WORLD, &rank);
10
11     if (0 == rank) {
12         MPI_Send(&rank, 1, MPI_INT, 1, 98, MPI_COMM_WORLD);
13         MPI_Recv(&recv, 1, MPI_INT, 1, 99, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
14     } else {
15         MPI_Send(&rank, 1, MPI_INT, 0, 99, MPI_COMM_WORLD);
16         MPI_Recv(&recv, 1, MPI_INT, 0, 98, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
17     }
18     std::cout << "Rank " << rank << " got ID " << recv << " from other rank\n";
19
20     MPI_Finalize();
21     return 0;
22 }
```

Try to predict what happens in the program on the left.

Is this code bug free?

Output

```
1 $ mpirun -n 2 ./mpi_p2p
2 Rank 0 got ID 1 from other rank
3 Rank 1 got ID 0 from other rank
```

Is the output on the left expected?

example-4

BLOCKING POINT-TO-POINT EXAMPLE

```
1  #include <mpi.h>
2  #include <iostream>
3
4  // exchange rank IDs between 2 processes
5  int main(int argc, char* argv[])
6  {
7      int rank, recv;
8      MPI_Init(&argc, &argv);
9      MPI_Comm_rank(MPI_COMM_WORLD, &rank);
10
11     if (0 == rank) {
12         MPI_Send(&rank, 1, MPI_INT, 1, 98, MPI_COMM_WORLD);
13         MPI_Recv(&recv, 1, MPI_INT, 1, 99, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
14     } else {
15         MPI_Send(&rank, 1, MPI_INT, 0, 99, MPI_COMM_WORLD);
16         MPI_Recv(&recv, 1, MPI_INT, 0, 98, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
17     }
18     std::cout << "Rank " << rank << " got ID " << recv << " from other rank\n";
19
20     MPI_Finalize();
21     return 0;
22 }
```

Try to predict what happens in the program on the left.

Is this code bug free?

No!

The (blocking) routines we use are non-local. The blocking `MPI_Send` will never be matched with a `MPI_Recv`. The execution will not make progress → **deadlock**. Yet, our program *did not* deadlock. Why?

MPI COMMUNICATION MODES

- *We were just lucky*. Sending a larger message would not have worked (or if many more processes were involved than just two).
- The procedure involved made use of what is called *standard mode*.
- In standard communication mode, MPI decides whether outgoing messages will be *buffered*. If MPI decides to buffer the message, the MPI_Send call may *complete* before a matching MPI_Recv is invoked. *This is the reason the code did not deadlock in the previous slide → the default buffer size in any MPI implementation is very small!*
- *Communication modes* allow you to choose the communication protocol.

MPI_Send = *standard mode*

- How standard mode behaves *is not defined in the standard* and therefore *implementation dependent*.
- Only *guarantee*: when the procedure returns to caller, the send buffer is *safe to be used*.
- MPI_Send is a *non-local* procedure. Successful completion may depend on the occurrence of a matching receive.

BLOCKING POINT-TO-POINT EXAMPLE (REVISITED)

example-4

```
1  #include <mpi.h>
2  #include <iostream>
3
4  // exchange rank IDs between 2 processes
5  int main(int argc, char* argv[])
6  {
7      int rank, recv;
8      MPI_Init(&argc, &argv);
9      MPI_Comm_rank(MPI_COMM_WORLD, &rank);
10
11     if (0 == rank) {
12         MPI_Send(&rank, 1, MPI_INT, 1, 98, MPI_COMM_WORLD);
13         MPI_Recv(&recv, 1, MPI_INT, 1, 99, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
14     } else {
15         MPI_Recv(&recv, 1, MPI_INT, 0, 98, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
16         MPI_Send(&rank, 1, MPI_INT, 0, 99, MPI_COMM_WORLD);
17     }
18     std::cout << "Rank " << rank << " got ID " << recv << " from other rank\n";
19
20     MPI_Finalize();
21     return 0;
22 }
```

The *order* with which you invoke MPI procedures matters.

The order of (blocking) MPI procedures matters! This is correct MPI code (deadlock free).

The MPI_Send used on rank 0 is matched with a MPI_Recv on rank 1 (both non-local procedures).
Rule: when you write MPI code, try to *post* the matching MPI_Recv of an MPI operation as early as possible.

COMPUTING π WITH BLOCKING POINT-TO-POINT

example-5

```
1 #include <iostream>
2 #include <mpi.h>
3
4 int main(int argc, char* argv[])
5 {
6     int rank, size;
7     MPI_Init(&argc, &argv);
8     MPI_Comm_rank(MPI_COMM_WORLD, &rank);
9     MPI_Comm_size(MPI_COMM_WORLD, &size);
10
11     const size_t n = 100000000;
12
13     // domain decomposition
14     const size_t chunk_size = (n + size - 1) / size;
15     const size_t my_start = rank * chunk_size;
16     const size_t my_end = std::min((rank + 1) * chunk_size, n);
17
18     // partial sum
19     double sum = 0.0; // rank local sum (local to each process)
20     for (size_t k = my_start; k < my_end; ++k) {
21         sum += (1.0 - 2.0 * (k % 2)) / (2.0 * k + 1.0);
22     }
23
24     // reduction
25     if (0 == rank) {
26         double pi = sum;
27         double recvp; // receive buffer
28         for (int r = 1; r < size; ++r) {
29             MPI_Recv(&recvp, 1, MPI_DOUBLE, r, 123,
30                     MPI_COMM_WORLD, MPI_STATUS_IGNORE);
31             pi += recvp;
32         }
33         std::cout << "pi = " << 4.0 * pi << '\n';
34     } else {
35         MPI_Send(&sum, 1, MPI_DOUBLE, 0, 123, MPI_COMM_WORLD);
36     }
37
38     MPI_Finalize();
39     return 0;
40 }
```

$$\Pi = 4 \sum_{k=0}^{\infty} \frac{(-1)^k}{2k+1}$$

- Let us compute Π with MPI this time. We need to divide the sum among MPI ranks instead of threads.
- There are no race conditions here!
- To reduce the partial sums, each rank needs to communicate its result with the root rank in the communicator (it could be any other rank as well).
- Note that the reduction phase involves as many steps as there are ranks. The way it is done here will scale badly if you have many ranks! It is a *collective* operation.

BLOCKING COLLECTIVE COMMUNICATION

Blocking collective communication routines:

- Commonly used operations that involve *all* ranks in a communicator.
- When a blocking collective procedure returns, it is safe to use the communication buffers again (*completion*).
- You cannot assume that ranks in the communicator are *synchronized* when a blocking collective procedure returns. The *barrier* is the only collective procedure that guarantees synchronization upon return.
- Some of the collective calls have a single *originating* or *receiving* rank. This rank is called *root*. Examples are MPI_Bcast (originating) or MPI_Reduce (receiving).
- Because communication is *collective*, *all* ranks must call the same collective primitive, otherwise *deadlock*.

Overview:

All to One:

- MPI_Reduce
- MPI_Gather

One to All:

- MPI_Bcast
- MPI_Scatter

All to All:

- MPI_Allgather
- MPI_Alltoall
- MPI_Allreduce
- MPI_Barrier

Other:

- MPI_Exscan
- MPI_Scan

BLOCKING COLLECTIVE COMMUNICATION

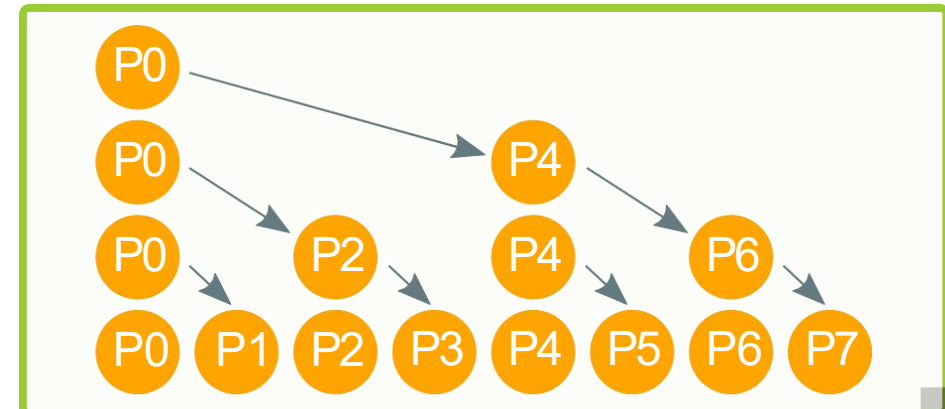
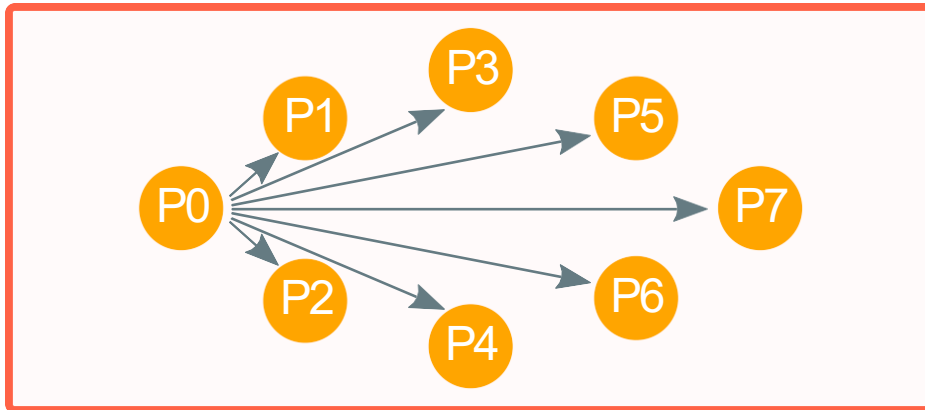
Why do we have collective procedures in the API:

- We can implement any collective with point-to-point procedures.
- Collectives exist because we might implement them poorly ourselves.
- MPI does not want you to implement your own (these are common operations). Instead, the MPI implementation *will pick the best automatically* depending on the *hardware* you are running on, the size of the communicator or due to algorithmic preference.

Example: MPI_Bcast (broadcast) – the root rank distributes data to all other ranks

Naive: the root rank must deal with 7 MPI_Send calls.
This will scale badly if many ranks are involved.

Log-step: in a tree based algorithm, root must deal only with $\log_2(n)$ MPI_Send calls, where n is the total number of ranks in the communicator. *Here other ranks help out broadcasting the data.*



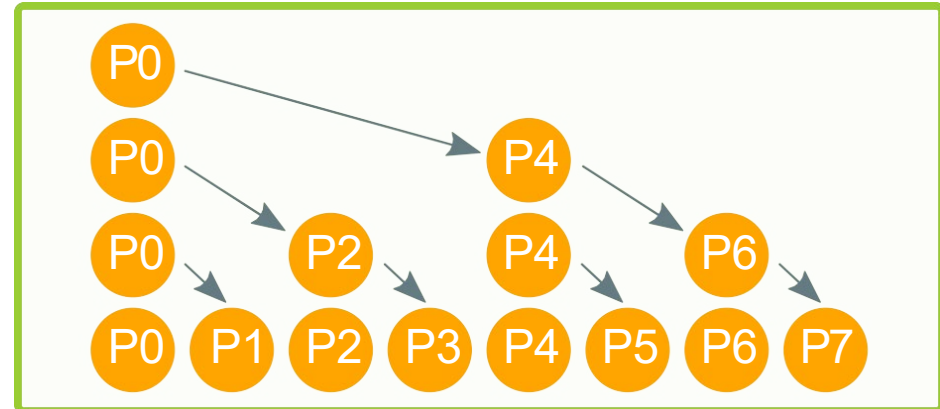
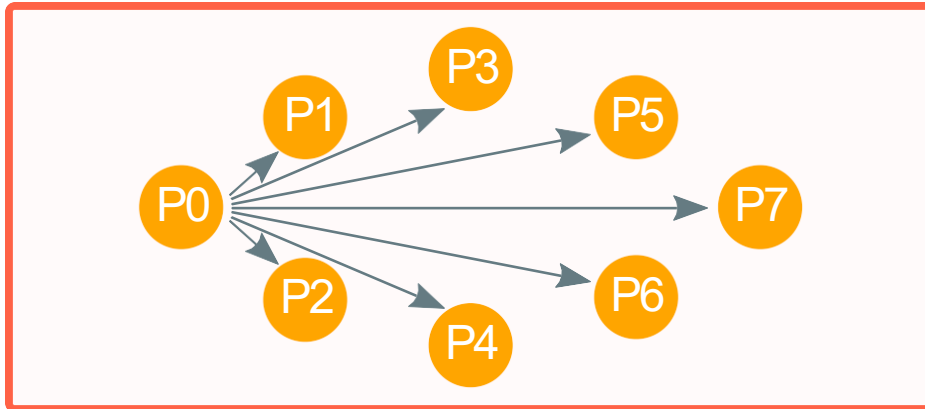
BLOCKING COLLECTIVE COMMUNICATION

Why do we have collective procedures in the API:

Example: MPI_Bcast (broadcast) — the root rank distributes data to all other ranks

Naive: the root rank must deal with 7 MPI_Send calls. This will scale badly if many ranks are involved.

Log-step: in a tree based algorithm, root must deal only with $\log_2(n)$ MPI_Send calls, where n is the total number of ranks in the communicator. *Here other ranks help out broadcasting the data.*



Note:

- For **8 ranks**: 3 versus 7 MPI_Send calls *may not* feel like a big improvement.
- For **1024 ranks**: 10 versus 1023 MPI_Send calls is a **100x** improvement!

REDUCTION OPERATION

Blocking MPI reduction:

```
1 int MPI_Reduce(const void *sendbuf,  
2               void *recvbuf,  
3               int count,  
4               MPI_Datatype datatype,  
5               MPI_Op op,  
6               int root,  
7               MPI_Comm comm)
```

sendbuf input data to reduce
recvbuf output result (*root* only)
count number of elements in sendbuf
datatype type of data elements
op reduction operator
root root rank ID
comm reduction communicator

- A reduction is a *collective* operation (**all** ranks must participate).
- You can reduce the data *in-place*. In this case recvbuf acts as *input-output* and sendbuf must be set to **MPI_IN_PLACE**. See man MPI_Reduce.

Basic MPI reduction operators:

MPI_Op	Meaning
MPI_MAX	maximum
MPI_MIN	minimum
MPI_SUM	sum
MPI_PROD	product
MPI_LAND	logical and
MPI_BAND	bit-wise and
MPI_LOR	logical or
MPI_BOR	bit-wise or
MPI_LXOR	logical exclusive or
MPI_BXOR	bit-wise exclusive or
MPI_MAXLOC	max value and location
MPI_MINLOC	min value and location

- The operation must be **associative**.
- The operation can be **commutative** but not required by the standard.
- You can define your own operator with **MPI_Op_create** (not covered in class).

COMPUTING π WITH BLOCKING REDUCTION

example-5

```
1 #include <iostream>
2 #include <mpi.h>
3
4 int main(int argc, char* argv[])
5 {
6     int rank, size;
7     MPI_Init(&argc, &argv);
8     MPI_Comm_rank(MPI_COMM_WORLD, &rank);
9     MPI_Comm_size(MPI_COMM_WORLD, &size);
10
11     const size_t n = 100000000;
12
13     // domain decomposition
14     const size_t chunk_size = (n + size - 1) / size;
15     const size_t my_start = rank * chunk_size;
16     const size_t my_end = std::min((rank + 1) * chunk_size, n);
17
18     // partial sum
19     double sum = 0.0; // rank local sum (local to each process)
20     for (size_t k = my_start; k < my_end; ++k) {
21         sum += (1.0 - 2.0 * (k % 2)) / (2.0 * k + 1.0);
22     }
23
24     // reduction
25     double pi = 0.0;
26     MPI_Reduce(&sum, &pi, 1, MPI_DOUBLE, MPI_SUM, 0, MPI_COMM_WORLD);
27     if (0 == rank) {
28         std::cout << "pi = " << 4.0 * pi << '\n';
29     }
30
31     MPI_Finalize();
32     return 0;
33 }
```

$$\Pi = 4 \sum_{k=0}^{\infty} \frac{(-1)^k}{2k+1}$$

- Less code and more clear!
- MPI takes care of the reduction for us.
- In this example the result is reduced to rank 0.
- All ranks must call MPI_Reduce otherwise the code will deadlock.

COMPUTING π WITH BLOCKING IN-PLACE REDUCTION

example-5

```
1 #include <iostream>
2 #include <mpi.h>
3
4 int main(int argc, char* argv[])
5 {
6     int rank, size;
7     MPI_Init(&argc, &argv);
8     MPI_Comm_rank(MPI_COMM_WORLD, &rank);
9     MPI_Comm_size(MPI_COMM_WORLD, &size);
10
11     const size_t n = 100000000;
12
13     // domain decomposition
14     const size_t chunk_size = (n + size - 1) / size;
15     const size_t my_start = rank * chunk_size;
16     const size_t my_end = std::min((rank + 1) * chunk_size, n);
17
18     // partial sum
19     double sum = 0.0; // rank local sum (local to each process)
20     for (size_t k = my_start; k < my_end; ++k) {
21         sum += (1.0 - 2.0 * (k % 2)) / (2.0 * k + 1.0);
22     }
23
24     // reduction
25     MPI_Reduce((0 == rank) ? MPI_IN_PLACE : &sum,
26               &sum, 1, MPI_DOUBLE, MPI_SUM, 0, MPI_COMM_WORLD);
27     if (0 == rank) {
28         std::cout << "pi = " << 4.0 * sum << '\n';
29     }
30
31     MPI_Finalize();
32     return 0;
33 }
```

$$\Pi = 4 \sum_{k=0}^{\infty} \frac{(-1)^k}{2k+1}$$

- You could save allocating another double by using the `MPI_IN_PLACE` handle.
- This is not always clearer.
- In this example, allocating one more double is not a big deal.



FIND MAX/MIN LOCATION WITH REDUCTION

example-6

```
1 #include <cstdlib>
2 #include <iostream>
3 #include <mpi.h>
4
5 int main(int argc, char* argv[])
6 {
7     int rank;
8     MPI_Init(&argc, &argv);
9     MPI_Comm_rank(MPI_COMM_WORLD, &rank);
10
11     srand48(rank);
12     const double value = drand48();
13     std::cout << "Rank " << rank << ": value = " << value << '\n';
14
15     // packed value and rank ID
16     struct DoubleInt {
17         double val;
18         int rank;
19     } send, recv;
20     send.val = value;
21     send.rank = rank;
22
23     MPI_Reduce(&send, &recv, 1, MPI_DOUBLE_INT,
24               MPI_MINLOC, 0, MPI_COMM_WORLD);
25     if (0 == rank) {
26         std::cout << "Minimum: value = " << recv.val;
27                 << " on rank " << recv.rank << '\n';
28     }
29
30     MPI_Finalize();
31     return 0;
32 }
```

- The `MPI_MAXLOC` and `MPI_MINLOC` reduce operators are used to determine a global maximum and/or minimum of a value including its location (on which rank).
- For this operation, MPI defines special data types that encode the value type as well as the index type (int). These are
 - `MPI_FLOAT_INT`
 - `MPI_DOUBLE_INT`
 - `MPI_LONG_INT`
 - `MPI_2INT`
 - `MPI_SHORT_INT`
 - `MPI_LONG_DOUBLE_INT`

See Section 6.9.3 in the MPI 4.0 standard.

Output:

```
1 $ mpirun -n 4 ./mpi_minloc
2 Rank 1: value = 0.0416303
3 Rank 2: value = 0.912433
4 Rank 3: value = 0.783235
5 Rank 0: value = 0.170828
6 Minimum: value = 0.0416303 on rank 1
```